

BÀI 14. PHÂN TÍCH DỮ LIỆU VỚI PYTHON - PHẦN 1 (NUMPY & PANDAS CƠ BẢN)

14.1. Giới thiệu Phân tích dữ liệu & NumPy

14.1.1. Khái niệm Phân tích dữ liệu

Phân tích dữ liệu là một quá trình khám phá, làm sạch, biến đổi và mô hình hóa dữ liệu với mục tiêu khám phá thông tin hữu ích, đưa ra kết luận, và hỗ trợ ra quyết định.

Mục tiêu chính của Phân tích dữ liệu:

- **Hiểu rõ dữ liệu:** Nắm bắt các đặc điểm, xu hướng, và mối quan hệ tiềm ẩn trong dữ liệu.
- **Giải quyết vấn đề:** Sử dụng dữ liệu để tìm ra nguyên nhân gốc rễ của một vấn đề hoặc cơ hội.
- **Dự đoán tương lai:** Xây dựng các mô hình dự đoán để dự báo các kết quả trong tương lai.
- **Hỗ trợ ra quyết định:** Cung cấp thông tin và cái nhìn sâu sắc để các cá nhân hoặc tổ chức đưa ra quyết định tốt hơn.

14.1.2. Quy trình cơ bản của Phân tích dữ liệu:

1. Thu thập dữ liệu (Data Collection):

- Lấy dữ liệu từ nhiều nguồn khác nhau như cơ sở dữ liệu, file CSV/Excel, API web, cảm biến, v.v.
- Đảm bảo dữ liệu phù hợp với câu hỏi nghiên cứu.

2. Làm sạch dữ liệu (Data Cleaning / Wrangling):

- Xử lý dữ liệu bị thiếu (missing values), trùng lặp (duplicates), hoặc không nhất quán (inconsistencies).
- Sửa lỗi chính tả, định dạng không đúng.
- Chuẩn hóa dữ liệu để đảm bảo tính đồng nhất.
- Đây thường là bước tốn thời gian nhất nhưng rất quan trọng.

3. Phân tích dữ liệu (Data Analysis):

- Áp dụng các kỹ thuật thống kê, thuật toán, và mô hình để khám phá các mẫu, xu hướng và mối quan hệ.
- Thực hiện các phép tính tổng hợp (sum, average, count), phân nhóm, tương quan, hồi quy, v.v.
- Kiểm tra giả thuyết và rút ra kết luận.

4. Trực quan hóa dữ liệu (Data Visualization):

- Biểu diễn kết quả phân tích dưới dạng biểu đồ, đồ thị, bản đồ, hoặc bảng điều khiển (dashboards).
- Giúp truyền đạt thông tin phức tạp một cách rõ ràng và dễ hiểu cho người không chuyên.
- Các công cụ phổ biến: Matplotlib, Seaborn, Plotly.

14.1.3. Vai trò của Python và các thư viện trong phân tích dữ liệu

Python đã trở thành ngôn ngữ hàng đầu trong lĩnh vực phân tích dữ liệu và khoa học dữ liệu nhờ vào:

- **Cú pháp dễ đọc và học:** Giúp các nhà phân tích tập trung vào logic thay vì phức tạp của ngôn ngữ.
- **Cộng đồng lớn và hỗ trợ mạnh mẽ:** Có rất nhiều tài nguyên, hướng dẫn và cộng đồng hỗ trợ.
- **Hệ sinh thái thư viện phong phú:** Đây là lý do chính. Các thư viện chuyên dụng giúp giải quyết mọi giai đoạn của quy trình phân tích dữ liệu.

14.1.4. Các thư viện Python chính cho phân tích dữ liệu:

- **NumPy (Numerical Python):** Nền tảng cho tính toán số học hiệu suất cao, đặc biệt với mảng đa chiều.
- **Pandas:** Xử lý và phân tích dữ liệu có cấu trúc (bảng) một cách hiệu quả. Được xây dựng trên NumPy.
- **Matplotlib / Seaborn:** Trực quan hóa dữ liệu, tạo biểu đồ.
- **Scikit-learn:** Học máy (Machine Learning) cho các tác vụ như phân loại, hồi quy, phân cụm.
- **SciPy:** Các công cụ khoa học và kỹ thuật (đại số tuyến tính, tối ưu hóa, xử lý tín hiệu).

14.1.5. Thư viện NumPy: Giới thiệu ndarray (mảng N chiều), ưu điểm so với list Python

NumPy (Numerical Python) là thư viện cơ bản cho tính toán khoa học trong Python. Nó cung cấp một đối tượng mảng hiệu suất cao là **ndarray (N-dimensional array)**.

ndarray là gì?

- Là một lưới các giá trị cùng kiểu dữ liệu, được lập chỉ mục bởi một bộ các số nguyên dương.
- Có thể là mảng 1 chiều (vector), 2 chiều (ma trận), hoặc N chiều (tensor).

Ưu điểm của ndarray so với list Python thông thường:

1. Tốc độ (Speed):

- NumPy arrays được triển khai bằng C và Fortran ở cấp độ thấp, giúp các phép toán trên mảng nhanh hơn đáng kể so với list Python (vốn là các đối tượng tổng quát hơn).
- Các phép toán được **vector hóa** (vectorized operations), nghĩa là chúng hoạt động trên toàn bộ mảng cùng lúc thay vì lặp qua từng phần tử.

2. Hiệu quả bộ nhớ (Memory Efficiency):

- ndarray lưu trữ các phần tử cùng kiểu dữ liệu liên tục trong bộ nhớ, tối ưu hóa việc sử dụng bộ nhớ.
- List Python lưu trữ các con trỏ đến các đối tượng riêng lẻ, có thể nằm rải rác trong bộ nhớ và tốn nhiều không gian hơn.

3. Chức năng mạnh mẽ:

- NumPy cung cấp hàng trăm hàm toán học và đại số tuyến tính được tối ưu hóa sẵn sàng để sử dụng trên mảng.

14.1.6. Tạo mảng NumPy, các thuộc tính của mảng (shape, dtype)

Cách tạo mảng NumPy:

1. Từ list Python: Sử dụng np.array()

```
import numpy as np

# Mảng 1 chiều (vector)
arr1d = np.array([1, 2, 3, 4, 5])
print(arr1d) # Output: [1 2 3 4 5]

# Mảng 2 chiều (ma trận)
```

```
arr2d = np.array([[1, 2, 3], [4, 5, 6]])
print(arr2d)

# Output:
# [[1 2 3]
#  [4 5 6]]
```

2. Sử dụng các hàm có sẵn của NumPy:

- `np.zeros(shape)`: Tạo mảng chứa toàn số 0.

```
zeros_arr = np.zeros((3, 4)) # Mảng 3x4 chứa số 0
print(zeros_arr)
```

- `np.ones(shape)`: Tạo mảng chứa toàn số 1.

```
ones_arr = np.ones((2, 2)) # Mảng 2x2 chứa số 1
print(ones_arr)
```

- `np.arange(start, stop, step)`: Tạo mảng với các giá trị trong một khoảng.

```
range_arr = np.arange(0, 10, 2) # [0 2 4 6 8]
print(range_arr)
```

- `np.linspace(start, stop, num)`: Tạo mảng với số lượng phần tử đều nhau trong một khoảng.

```
lin_arr = np.linspace(0, 1, 5) # 5 số từ 0 đến 1, đều nhau: [0.  0.25
0.5  0.75 1.  ]
print(lin_arr)
```

- `np.random.rand(d0, d1, ..., dn)`: Tạo mảng với các số ngẫu nhiên từ phân phối đều (uniform distribution) giữa 0 và 1.

```
rand_arr = np.random.rand(2, 3) # Mảng 2x3 số ngẫu nhiên
print(rand_arr)
```

- `np.random.randint(low, high, size)`: Tạo mảng với các số nguyên ngẫu nhiên trong một khoảng.

```
rand_int_arr = np.random.randint(0, 10, size=(2, 2)) # Mảng 2x2 số nguyên
ngẫu nhiên từ 0 đến 9
print(rand_int_arr)
```

Các thuộc tính quan trọng của mảng NumPy:

- **shape**: Một tuple biểu diễn kích thước của mảng theo từng chiều.

Python

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
print(arr.shape) # Output: (2, 3) (2 hàng, 3 cột)
```

```
arr1d = np.array([1, 2, 3])
```

```
print(arr1d.shape) # Output: (3,) (3 phần tử, 1 chiều)
```

- **dtype**: Kiểu dữ liệu của các phần tử trong mảng (ví dụ: int32, float64). Tất cả các phần tử trong ndarray phải có cùng kiểu dữ liệu.

```
arr = np.array([1, 2, 3])
print(arr.dtype) # Output: int64 (hoặc int32 tùy hệ thống)

arr_float = np.array([1.0, 2.5, 3.0])
print(arr_float.dtype) # Output: float64
```

- **ndim**: Số chiều của mảng.

```
arr = np.array([[1, 2], [3, 4]])
print(arr.ndim) # Output: 2
```

- **size**: Tổng số phần tử trong mảng.

```
arr = np.array([[1, 2], [3, 4]])
print(arr.size) # Output: 4
```

14.2. Các phép toán NumPy & Pandas Series

14.2.1. Các phép toán số học trên mảng NumPy (element-wise operations)

Một trong những ưu điểm lớn nhất của NumPy là khả năng thực hiện các phép toán số học trên toàn bộ mảng một cách nhanh chóng và hiệu quả, được gọi là **element-wise operations**. Điều này có nghĩa là phép toán được áp dụng cho từng cặp phần tử tương ứng trong các mảng.

```
import numpy as np

arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])

# Cộng
result_add = arr1 + arr2
```

```
print(f"Cộng: {result_add}") # Output: [5 7 9]

# Trừ
result_sub = arr1 - arr2
print(f"Trừ: {result_sub}") # Output: [-3 -3 -3]

# Nhân (element-wise multiplication, KHÔNG phải nhân ma trận)
result_mul = arr1 * arr2
print(f"Nhân: {result_mul}") # Output: [ 4 10 18]

# Chia
result_div = arr1 / arr2
print(f"Chia: {result_div}") # Output: [0.25 0.4 0.5 ]

# Lũy thừa
result_pow = arr1 ** 2
print(f"Lũy thừa: {result_pow}") # Output: [1 4 9]

# Các hàm toán học khác
print(f"Sin của arr1: {np.sin(arr1)}")
print(f"Log của arr1: {np.log(arr1)}")
```

Lưu ý: Để thực hiện các phép toán trên hai mảng, chúng phải có kích thước tương thích (cùng shape hoặc có thể được "broadcast" - sẽ học sau).

14.2.2. Lập chỉ mục (indexing) và cắt (slicing) mảng NumPy

Tương tự như list Python, bạn có thể truy cập các phần tử hoặc các phần của mảng NumPy bằng lập chỉ mục và cắt.

Lập chỉ mục (Indexing):

- Sử dụng ngoặc vuông []. Bắt đầu từ 0.
- Với mảng đa chiều, sử dụng dấu phẩy để phân cách các chỉ số cho từng chiều.

```
arr1d = np.array([10, 20, 30, 40, 50])
print(f"Phần tử thứ 2: {arr1d[1]}") # Output: 20
print(f"Phần tử cuối cùng: {arr1d[-1]}") # Output: 50

arr2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(f"Phần tử ở hàng 1, cột 2: {arr2d[0, 1]}") # Output: 2 (hàng 0, cột 1)
print(f"Hàng thứ 2: {arr2d[1]}") # Output: [4 5 6]
```

Cắt (Slicing):

- Sử dụng cú pháp [start:stop:step]. stop không bao gồm.
- Có thể bỏ qua start, stop, step để lấy giá trị mặc định.

```
arr1d = np.array([10, 20, 30, 40, 50, 60])
print(f"Cắt từ 1 đến 4: {arr1d[1:4]}") # Output: [20 30 40]
print(f"Cắt từ đầu đến 3: {arr1d[:3]}") # Output: [10 20 30]
print(f"Cắt từ 2 đến cuối: {arr1d[2:]}") # Output: [30 40 50 60]
print(f"Cắt với bước nhảy 2: {arr1d[:2]}") # Output: [10 30 50]

arr2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(f"Cắt hàng 0, cột 1 và 2: {arr2d[0, 1:3]}") # Output: [2 3]
print(f"Cắt tất cả hàng, cột cuối cùng: {arr2d[:, 2]}") # Output: [3 6 9]
(lấy tất cả hàng, cột chỉ số 2)
print(f"Cắt các hàng 0 và 1, tất cả cột: {arr2d[0:2, :]}")
```

Output:

```
# [[1 2 3]
#  [4 5 6]]
```

14.2.3. Thư viện Pandas: Giới thiệu Series (cấu trúc dữ liệu 1 chiều, giống mảng có nhãn)

Pandas là một thư viện Python mạnh mẽ để phân tích và thao tác dữ liệu. Nó được xây dựng trên NumPy và cung cấp các cấu trúc dữ liệu linh hoạt, dễ sử dụng, phù hợp với dữ liệu bảng hoặc dữ liệu có cấu trúc.

Series là gì?

- **Series** là cấu trúc dữ liệu **một chiều** trong Pandas.
- Nó có thể được coi là một mảng NumPy có gắn nhãn (label) cho mỗi phần tử, gọi là **index**.
- Các phần tử trong Series có thể thuộc bất kỳ kiểu dữ liệu nào (integer, float, string, Python object, v.v.).

Ví dụ một Series:

```
a    10
b    20
c    30
dtype: int64
```

- Ở đây, a, b, c là các nhãn (index), và 10, 20, 30 là các giá trị.

Tạo Series từ List, Dictionary

Cách tạo Series:

1. Từ một List (mặc định index là số nguyên từ 0):

```
import pandas as pd
data_list = [10, 20, 30, 40, 50]
s1 = pd.Series(data_list)
print(s1)
```

Output:

```
# 0    10
# 1    20
# 2    30
# 3    40
# 4    50
# dtype: int64
```

2. Từ một List với Index tùy chỉnh:

```
data_list = [10, 20, 30]
custom_index = ['a', 'b', 'c']
s2 = pd.Series(data_list, index=custom_index)
print(s2)
```

Output:

```
# a    10
# b    20
# c    30
# dtype: int64
```

3. Từ một Dictionary (keys trở thành index, values trở thành data):

```
data_dict = {'Math': 90, 'Science': 85, 'English': 92}
s3 = pd.Series(data_dict)
print(s3)
```

Output:

```
# Math      90
# Science   85
# English   92
# dtype: int64
```


Truy cập phần tử trong Series:

Bạn có thể truy cập phần tử bằng vị trí số nguyên hoặc bằng nhãn index.

```
print(s1[0])          # Output: 10 (bằng vị trí)
print(s2['b'])         # Output: 20 (bằng nhãn)
print(s3['Math'])      # Output: 90 (bằng nhãn)
# Cắt (slicing)
print(s1[1:4])
```

Output:

```
# 1      20
# 2      30
# 3      40
# dtype: int64
# Lọc
print(s3[s3 > 90])
```

Output:

```
# English      92
# dtype: int64
```

14.2.4. Pandas DataFrame & Đọc/Ghi dữ liệu

DataFrame: Cấu trúc dữ liệu 2 chiều, giống bảng tính (có hàng, cột)

DataFrame là cấu trúc dữ liệu quan trọng nhất trong Pandas, đại diện cho dữ liệu dạng **bảng hai chiều** (two-dimensional, tabular data), giống như một bảng tính Excel hoặc một bảng trong cơ sở dữ liệu.

- Mỗi cột trong DataFrame là một đối tượng **Series**.
- Mỗi cột có thể chứa các kiểu dữ liệu khác nhau (số, chuỗi, boolean, v.v.).
- DataFrame có cả chỉ mục hàng (row index) và chỉ mục cột (column index hay column names).

Ví dụ một DataFrame:

	Name	Age	City
0	Bim	23	HCM
1	Bo	20	TG
2	Bin	15	HN

- Ở đây, 0, 1, 2 là chỉ mục hàng, và Name, Age, City là chỉ mục cột (tên cột).

Tạo DataFrame từ Dictionary, List of Lists

Cách tạo DataFrame:

1. Từ một Dictionary của các List (phổ biến nhất):

- Các key của dictionary sẽ trở thành tên cột.
- Các value là các list chứa dữ liệu cho từng cột.
- Đảm bảo tất cả các list có cùng độ dài.

```
import pandas as pd

data = {
    'Name': ['Bim', 'Bo', 'Bin'],
    'Age': [13, 20, 15],
    'City': ['Ho Chi Minh', 'London', 'Paris']
}
df1 = pd.DataFrame(data)
print(df1)
```

2. Từ một List of Lists (phải cung cấp tên cột):

```
data = [
    ['Bim', 13, 'HCM'],
    ['Bo', 20, 'TG'],
    ['Bin', 35, 'HN']
]
columns = ['Name', 'Age', 'City']
df2 = pd.DataFrame(data, columns=columns)
print(df2)
```

3. Từ một List of Dictionaries (mỗi dictionary là một hàng):

```
data = [
    {'Name': 'Bim', 'Age': 15, 'City': 'HCM'},
    {'Name': 'Bo', 'Age': 20, 'City': 'TG'},
    {'Name': 'Bin', 'Age': 15, 'City': 'HN'}
]
df3 = pd.DataFrame(data)
print(df3)
```

14.2.5. Đọc dữ liệu từ các file: `pd.read_csv()`, `pd.read_excel()`

Pandas cung cấp các hàm rất tiện lợi để đọc dữ liệu từ nhiều định dạng file khác nhau.

- **pd.read_csv(filepath_or_buffer, ...)**: Đọc dữ liệu từ file CSV (Comma Separated Values).
 - filepath_or_buffer: Đường dẫn đến file.
 - sep: Ký tự phân cách (mặc định là ,).
 - header: Số hàng dùng làm tiêu đề cột (mặc định là 0, hàng đầu tiên).
 - index_col: Cột dùng làm chỉ mục hàng.
 - skiprows: Số hàng bỏ qua từ đầu file.
 - nrows: Số hàng cần đọc.
 - encoding: Mã hóa file (ví dụ: 'utf-8', 'latin1').

```
# df = pd.read_csv('data.csv')
# df = pd.read_csv('data_with_tabs.txt', sep='\t')
```

- **pd.read_excel(io, sheet_name=0, ...)**: Đọc dữ liệu từ file Excel (.xls, .xlsx).
 - io: Đường dẫn đến file.
 - sheet_name: Tên hoặc chỉ số của sheet cần đọc (mặc định là 0, sheet đầu tiên).

```
# df = pd.read_excel('sales_data.xlsx')
# df = pd.read_excel('multi_sheet.xlsx', sheet_name='Sheet2')
```

14.2.6. Ghi dữ liệu ra file: df.to_csv(), df.to_excel()

Bạn cũng có thể dễ dàng ghi DataFrame trở lại các định dạng file.

- **df.to_csv(path_or_buf, index=True, header=True, ...)**: Ghi DataFrame ra file CSV.
 - path_or_buf: Đường dẫn file đầu ra.
 - index=False: Không ghi chỉ mục hàng vào file (rất phổ biến để tránh tạo ra một cột chỉ mục thừa).
 - header=False: Không ghi tiêu đề cột vào file.

```
# df.to_csv('output.csv', index=False) # Ghi ra file CSV không kèm index
```

- **df.to_excel(excel_writer, sheet_name='Sheet1', index=True, ...)**: Ghi DataFrame ra file Excel.
 - excel_writer: Đường dẫn file đầu ra.
 - sheet_name: Tên của sheet.

```
# df.to_excel('output.xlsx', index=False, sheet_name='Processed Data')
```

14.2.7. Các phương thức xem thông tin cơ bản của DataFrame: head(), tail(), info(), describe()

Sau khi đọc dữ liệu vào DataFrame, việc đầu tiên bạn muốn làm là khám phá cấu trúc và nội dung của nó.

– **df.head(n=5):**

```
Hiển thị n hàng đầu tiên của DataFrame (mặc định là 5).  
Giúp bạn có cái nhìn nhanh về dữ liệu.  
# print(df.head())  
# print(df.head(10))
```

– **df.tail(n=5):**

- Hiển thị **n hàng cuối cùng** của DataFrame (mặc định là 5).
- Hữu ích để xem các bản ghi mới nhất hoặc kiểm tra dữ liệu ở cuối file.

```
# print(df.tail())
```

– **df.info():**

- In ra một bản tóm tắt ngắn gọn về DataFrame.
- Bao gồm số lượng hàng, số lượng cột, số lượng giá trị không bị thiếu (Non-Null Count) cho mỗi cột, kiểu dữ liệu (Dtype) của mỗi cột, và thông tin sử dụng bộ nhớ.
- Rất hữu ích để kiểm tra nhanh các cột có giá trị thiếu.

```
# df.info()
```

– **df.describe():**

- Tạo ra các **thống kê mô tả** tóm tắt cho các cột số học trong DataFrame.
- Bao gồm count (số lượng giá trị không thiếu), mean (giá trị trung bình), std (độ lệch chuẩn), min (giá trị nhỏ nhất), 25% (bách phân vị thứ 25), 50% (trung vị/bách phân vị thứ 50), 75% (bách phân vị thứ 75), max (giá trị lớn nhất).
- Giúp bạn hiểu phân phối của dữ liệu số.

```
# print(df.describe())
```

THỰC HÀNH 14: NUMPY & PANDAS

14.1. Làm quen với NumPy

14.1.1. Cài đặt thư viện NumPy

Mục tiêu: Chuẩn bị môi trường để sử dụng NumPy.

Các bước thực hành:

1. Mở Terminal (hoặc Command Prompt trên Windows).
2. Nếu bạn đang ở trong một môi trường ảo (như đã học trong các tuần trước), hãy kích hoạt nó.
3. Chạy lệnh cài đặt:

```
pip install numpy
```

4. Để kiểm tra xem NumPy đã được cài đặt thành công chưa, bạn có thể mở một trình thông dịch Python (gõ python hoặc python3 vào Terminal) và thử import:

```
import numpy as np
print(np.__version__)
# Nếu không có lỗi và hiển thị phiên bản (ví dụ: 1.26.4) là thành công.
```

- Gõ exit() để thoát khỏi trình thông dịch.

14.2. Thực hành tạo các mảng NumPy 1D, 2D từ list và sử dụng np.zeros(), np.ones(), np.random.rand()

Mục tiêu: Tạo các loại mảng NumPy khác nhau.

Các bước thực hành:

1. Mở một file Python mới (ví dụ: numpy_intro.py) trong trình soạn thảo code của bạn.
2. Viết code sau vào file:

```
import numpy as np

print("--- Tạo mảng 1D từ list ---")
list_1d = [1, 2, 3, 4, 5]
arr_1d = np.array(list_1d)
print(f"Mảng 1D: {arr_1d}")
print(f"Kích thước (shape): {arr_1d.shape}")
print(f"Kiểu dữ liệu (dtype): {arr_1d.dtype}")
```

```
print(f"Số chiều (ndim): {arr_1d.ndim}")
print(f"Tổng số phần tử (size): {arr_1d.size}\n")

print("---- Tạo mảng 2D từ list các list ----")
list_2d = [[10, 11, 12], [13, 14, 15]]
arr_2d = np.array(list_2d)
print(f"Mảng 2D:\n{arr_2d}")
print(f"Kích thước (shape): {arr_2d.shape}")
print(f"Kiểu dữ liệu (dtype): {arr_2d.dtype}")
print(f"Số chiều (ndim): {arr_2d.ndim}")
print(f"Tổng số phần tử (size): {arr_2d.size}\n")

print("---- Tạo mảng toàn số 0 với np.zeros() ----")
zeros_matrix = np.zeros((3, 2)) # Mảng 3 hàng, 2 cột
print(f"Ma trận 0:\n{zeros_matrix}\n")

print("---- Tạo mảng toàn số 1 với np.ones() ----")
ones_vector = np.ones(5) # Mảng 1D với 5 phần tử
print(f"Vector 1: {ones_vector}\n")

print("---- Tạo mảng số ngẫu nhiên từ 0 đến 1 với np.random.rand() ----")
random_matrix = np.random.rand(2, 4) # Mảng 2 hàng, 4 cột
print(f"Ma trận ngẫu nhiên:\n{random_matrix}\n")

print("---- Tạo mảng số nguyên ngẫu nhiên với np.random.randint() ----")
random_integers = np.random.randint(1, 100, size=(3, 3)) # Số nguyên từ 1 đến 99, kích thước 3x3
print(f"Ma trận số nguyên ngẫu nhiên:\n{random_integers}\n")

print("---- Tạo mảng rỗng với np.empty() (chứa giá trị rác) ----")
empty_arr = np.empty((2, 2))
print(f"Mảng rỗng:\n{empty_arr}\n")
```

3. Lưu file và chạy nó từ Terminal: `python numpy_intro.py`.

4. Quan sát output và so sánh với lý thuyết đã học.

14.2.1. Thực hiện các phép toán số học cơ bản trên mảng (cộng, trừ, nhân ma trận)

Mục tiêu: Áp dụng các phép toán element-wise và nhân ma trận.

Các bước thực hành:

1. Tiếp tục chỉnh sửa file `numpy_intro.py` hoặc tạo file mới.
2. Thêm đoạn code sau:

```
import numpy as np

print("--- Các phép toán số học element-wise ---")
arr_a = np.array([1, 2, 3])
arr_b = np.array([4, 5, 6])

print(f"arr_a: {arr_a}")
print(f"arr_b: {arr_b}")
print(f"arr_a + arr_b = {arr_a + arr_b}")
print(f"arr_a - arr_b = {arr_a - arr_b}")
print(f"arr_a * arr_b (element-wise) = {arr_a * arr_b}")
print(f"arr_a / arr_b = {arr_a / arr_b}")
print(f"arr_a ** 2 = {arr_a ** 2}\n")

print("--- Phép toán trên mảng 2D ---")
matrix_a = np.array([[1, 2], [3, 4]])
matrix_b = np.array([[5, 6], [7, 8]])

print(f"Matrix A:\n{matrix_a}")
print(f"Matrix B:\n{matrix_b}")
print(f"Matrix A + Matrix B:\n{matrix_a + matrix_b}\n")

print("--- Nhân ma trận (dot product) ---")
# Sử dụng np.dot() hoặc toán tử @ (Python 3.5+)
matrix_c = np.array([[1, 2], [3, 4]])
matrix_d = np.array([[5, 6], [7, 8]])

dot_product_result = np.dot(matrix_c, matrix_d)
# Hoặc: dot_product_result = matrix_c @ matrix_d (chỉ với Python 3.5 trở
lên)
print(f"Matrix C:\n{matrix_c}")
print(f"Matrix D:\n{matrix_d}")
print(f"Kết quả nhân ma trận (C @ D):\n{dot_product_result}\n")

print("--- Lập chỉ mục và cắt mảng 1D ---")
arr_1d_slice = np.array([10, 20, 30, 40, 50, 60, 70])
print(f"Mảng: {arr_1d_slice}")
```

```
print(f"Phần tử thứ 3 (chỉ số 2): {arr_1d_slice[2]}")
print(f"Cắt từ chỉ số 1 đến 4 (không bao gồm 4): {arr_1d_slice[1:4]}")
print(f"Cắt với bước nhảy 2: {arr_1d_slice[::2]}\n")

print("--- Lập chỉ mục và cắt mảng 2D ---")
arr_2d_slice = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])
print(f"Mảng 2D:\n{arr_2d_slice}")
print(f"Phần tử ở hàng 1, cột 2 (chỉ số 0,1): {arr_2d_slice[0, 1]}")
print(f"Hàng thứ 2 (chỉ số 1): {arr_2d_slice[1, :]}") # Hoặc đơn giản:
arr_2d_slice[1]
print(f"Tất cả hàng, cột 3 (chỉ số 2): {arr_2d_slice[:, 2]}")
print(f"Cắt một khối con (hàng 0-1, cột 0-1):\n{arr_2d_slice[0:2,
0:2]}\n")
```

3. Lưu file và chạy nó. Quan sát các kết quả và tự mình thử nghiệm với các chỉ số và phạm vi cắt khác nhau.

14.3. Làm quen với Pandas Series & DataFrame

14.3.1. Cài đặt thư viện Pandas

Mục tiêu: Chuẩn bị môi trường để sử dụng Pandas.

Các bước thực hành:

1. Mở Terminal (hoặc Command Prompt trên Windows).
2. Đảm bảo môi trường ảo đang được kích hoạt.
3. Chạy lệnh cài đặt:

```
pip install pandas
```

4. Kiểm tra: Mở trình thông dịch Python và thử `import pandas as pd;`
`print(pd.__version__)`.

14.3.2. Thực hành tạo Series từ list và dictionary

Mục tiêu: Tạo các đối tượng Series cơ bản.

Các bước thực hành:

1. Mở một file Python mới (ví dụ: `pandas_intro.py`).

2. Viết code sau:

```
import pandas as pd
import numpy as np

print("--- Tạo Series từ List ---")
data_list = [10, 20, 30, 40, 50]
s_from_list = pd.Series(data_list)
print(f"Series từ list:\n{s_from_list}\n")

print("--- Tạo Series từ List với Index tùy chỉnh ---")
fruits = ['Apple', 'Banana', 'Cherry']
counts = [100, 150, 75]
s_fruits = pd.Series(counts, index=fruits)
print(f"Series trái cây:\n{s_fruits}\n")

print("--- Tạo Series từ Dictionary ---")
grades_dict = {'Math': 95, 'Science': 88, 'English': 92, 'History': 78}
s_grades = pd.Series(grades_dict)
print(f"Series điểm số:\n{s_grades}\n")

print("--- Truy cập phần tử trong Series ---")
print(f"Phần tử đầu tiên của s_from_list (chỉ số 0): {s_from_list[0]}")
print(f"Số chuỗi (Banana) trong s_fruits: {s_fruits['Banana']}")
print(f"Điểm môn Toán (Math) trong s_grades: {s_grades['Math']}\n")

print("--- Lọc và Cắt Series ---")
print(f"Điểm số trên 90:\n{s_grades[s_grades > 90]}\n")
print(f"Hai phần tử đầu tiên của s_fruits:\n{s_fruits[:2]}\n")
```

3. Lưu và chạy file.

14.3.3. Thực hành tạo DataFrame từ dictionary hoặc list of dictionaries

Mục tiêu: Tạo các đối tượng DataFrame cơ bản.

Các bước thực hành:

1. Tiếp tục chỉnh sửa file pandas_intro.py hoặc tạo file mới.
2. Thêm đoạn code sau:

```
import pandas as pd
import numpy as np
```

```
print("--- Tạo DataFrame từ Dictionary của các List/Arrays ---")
data_dict = {
    'Student': ['Bim', 'Bo', 'Bin', 'Bi'],
    'Age': [20, 22, 21, 20],
    'Major': ['CS', 'Math', 'Physics', 'CS'],
    'GPA': [3.8, 3.5, 3.9, 3.7]
}
df_students = pd.DataFrame(data_dict)
print(f"DataFrame sinh viên:\n{df_students}\n")

print("--- Tạo DataFrame từ List of Dictionaries ---")
records = [
    {'Product': 'Laptop', 'Price': 1200, 'Quantity': 50},
    {'Product': 'Mouse', 'Price': 25, 'Quantity': 200},
    {'Product': 'Keyboard', 'Price': 75, 'Quantity': 100}
]
df_products = pd.DataFrame(records)
print(f"DataFrame sản phẩm:\n{df_products}\n")

print("--- Truy cập cột trong DataFrame ---")
print(f"Cột 'Student':\n{df_students['Student']}\n") # Trả về Series
print(f"Cột 'Price' (cách 2):\n{df_products.Price}\n") # Truy cập bằng
dấu chấm (nếu tên cột không có khoảng trắng)

print("--- Truy cập hàng trong DataFrame bằng iloc (theo vị trí số
nguyên) ---")
print(f"Hàng đầu tiên (chỉ số 0):\n{df_students.iloc[0]}\n")
print(f"Hàng 1 và 2:\n{df_students.iloc[1:3]}\n")

print("--- Truy cập hàng trong DataFrame bằng loc (theo nhãn index) ---")
# Thay đổi index để thấy rõ loc
df_students_indexed = df_students.set_index('Student')
print(f"DataFrame sinh viên với index là tên:\n{df_students_indexed}\n")
print(f"Dữ liệu của Bo:\n{df_students_indexed.loc['Bo']}\n")
print(f"Dữ liệu của Bim và Bin:\n{df_students_indexed.loc[['Bim',
'Bin']]}\n")

print("--- Lọc DataFrame theo điều kiện ---")
# Lọc sinh viên có GPA > 3.7
high_gpa_students = df_students[df_students['GPA'] > 3.7]
```

```
print(f"Sinh viên GPA cao:\n{high_gpa_students}\n")

# Lọc sản phẩm có giá < 100 và số lượng > 50
affordable_popular_products = df_products[(df_products['Price'] < 100) &
(df_products['Quantity'] > 50)]
print(f"Sản phẩm giá tốt & phổ biến:\n{affordable_popular_products}\n")
```

3. Lưu và chạy file.

14.4. Đọc/Ghi & Khám phá dữ liệu

14.4.1. Đọc một tập dữ liệu công khai (.csv hoặc .xlsx) vào DataFrame (ví dụ: dữ liệu về dân số, doanh số bán hàng, thời tiết)

Mục tiêu: Lấy dữ liệu từ file thực tế vào Pandas để phân tích.

Các bước thực hành:

1. Tải một tập dữ liệu mẫu:

- Truy cập các nguồn dữ liệu công khai như Kaggle, data.gov, hoặc đơn giản hơn, tìm kiếm "sample csv data" trên Google.
- **Ví dụ:** Bạn có thể tải file titanic.csv từ trang Kaggle (nếu bạn có tài khoản) hoặc tìm kiếm "titanic.csv sample" để tải một phiên bản đơn giản.
- Hoặc tự tạo một file sales_data.csv đơn giản bằng trình soạn thảo văn bản:

```
Date,Region,Product,Sales,Units
2024-01-01,East,Laptop,12000,10
2024-01-01,West,Mouse,500,20
2024-01-02,East,Keyboard,1500,15
2024-01-02,Central,Laptop,9000,8
2024-01-03,West,Monitor,3000,5
```

- Lưu file này vào cùng thư mục với file Python của bạn (hoặc cung cấp đường dẫn đầy đủ).

2. Đọc dữ liệu vào DataFrame:

- Mở file Python mới (ví dụ: data_exploration.py).
- Viết code:

```
import pandas as pd

# Đảm bảo file sales_data.csv nằm cùng thư mục với file .py này
```

```
# hoặc cung cấp đường dẫn đầy đủ:
pd.read_csv('path/to/your/sales_data.csv')
try:
    df_sales = pd.read_csv('sales_data.csv')
    print("Đã đọc file sales_data.csv thành công.\n")
    print(df_sales)
except FileNotFoundError:
    print("Lỗi: Không tìm thấy file 'sales_data.csv'. Đảm bảo file nằm cùng thư mục hoặc cung cấp đúng đường dẫn.")

# Ví dụ đọc file Excel (nếu bạn có file .xlsx)
# try:
#     df_excel = pd.read_excel('your_excel_file.xlsx',
# sheet_name='Sheet1')
#     print("\nĐã đọc file Excel thành công.\n")
#     print(df_excel.head())
# except FileNotFoundError:
#     print("\nLỗi: Không tìm thấy file Excel. Bỏ qua nếu bạn không có file .xlsx.")
```

3. Lưu và chạy file.

14.4.2. Thực hành ghi một DataFrame ra file CSV/Excel

Mục tiêu: Lưu kết quả phân tích hoặc dữ liệu đã được xử lý ra file.

Các bước thực hành:

1. Tiếp tục file data_exploration.py.
2. Giả sử bạn đã đọc df_sales. Bây giờ, hãy thêm code để ghi nó ra một file mới.

```
import pandas as pd

# Đọc file (giả sử đã chạy thành công ở bước trước)
try:
    df_sales = pd.read_csv('sales_data.csv')
    print("\nDataFrame gốc:")
    print(df_sales)

    # Thêm một cột mới (ví dụ: Doanh thu trung bình mỗi đơn vị)
    df_sales['Avg_Revenue_Per_Unit'] = df_sales['Sales'] /
df_sales['Units']

    # Ghi DataFrame đã xử lý ra một file CSV mới
```

```
output_csv_file = 'processed_sales_data.csv'
df_sales.to_csv(output_csv_file, index=False) # index=False để không ghi cột index của DataFrame
print(f"\nDataFrame đã được ghi ra '{output_csv_file}' (không có cột index).")

# Ghi ra file Excel
output_excel_file = 'processed_sales_data.xlsx'
df_sales.to_excel(output_excel_file, index=False, sheet_name='Sales Report')
print(f"DataFrame đã được ghi ra '{output_excel_file}' (không có cột index).\n")

except FileNotFoundError:
    print("Không thể ghi file vì 'sales_data.csv' không tìm thấy.")
Luu và chạy file. Kiểm tra thư mục của bạn, bạn sẽ thấy các file processed_sales_data.csv và processed_sales_data.xlsx mới được tạo. Mở chúng bằng trình soạn thảo văn bản hoặc Excel để xem nội dung.
```

14.4.3. Sử dụng head(), info(), describe() để khám phá dữ liệu ban đầu

Mục tiêu: Thực hiện các bước khám phá dữ liệu cơ bản.

Các bước thực hành:

1. Tiếp tục file data_exploration.py.
2. Sau khi đọc file sales_data.csv vào df_sales, thêm các lệnh sau để khám phá:

```
import pandas as pd

try:
    df_sales = pd.read_csv('sales_data.csv')
    print("--- Khám phá dữ liệu sales_data.csv ---")

    print("\n5 hàng đầu tiên (df.head()):")
    print(df_sales.head())

    print("\n3 hàng cuối cùng (df.tail(3)):")
    print(df_sales.tail(3))

    print("\nThông tin tổng quan về DataFrame (df.info()):")
    df_sales.info()
```

```
print("\nThống kê mô tả cho các cột số (df.describe()):")
print(df_sales.describe())

# Kiểm tra kiểu dữ liệu của từng cột
print("\nKiểu dữ liệu của từng cột (df.dtypes):")
print(df_sales.dtypes)

# Lấy tên cột
print("\nTên các cột (df.columns):")
print(df_sales.columns.tolist()) # Chuyển đổi thành list để dễ nhìn

# Lấy kích thước DataFrame (số hàng, số cột)
print(f"\nKích thước DataFrame (hàng, cột): {df_sales.shape}")

except FileNotFoundError:
    print("Không thể khám phá dữ liệu vì 'sales_data.csv' không tìm thấy.")
```

3. Lưu và chạy file. Phân tích kết quả đầu ra:

- `head()` và `tail()` cho bạn cái nhìn trực quan về dữ liệu.
- `info()` cho bạn biết có bao nhiêu hàng, tên cột, kiểu dữ liệu và quan trọng nhất là số lượng giá trị không thiếu (Non-Null Count). Nếu Non-Null Count ít hơn tổng số hàng, có nghĩa là có dữ liệu bị thiếu.
- `describe()` cung cấp các thống kê tổng hợp cho các cột số, giúp bạn hiểu về phân bố, giá trị trung bình, min/max, và độ lệch chuẩn.

BÀI TẬP:

1. Đọc file dữ liệu. Hiển thị thông tin tổng quan (`info()`, `describe()`) và kiểm tra kiểu dữ liệu của các cột.
2. Sử dụng Pandas Series để tính toán tỷ lệ phần trăm của từng danh mục trong một cột phân loại (ví dụ: cột 'Region').
3. Tạo một DataFrame từ Dictionary. Thêm một cột mới 'Profit' bằng cách lấy giá trị từ cột 'Sales' trừ đi cột 'Cost' (phép toán Vectorized).

BÀI 15. PHÂN TÍCH DỮ LIỆU VỚI PYTHON - PHẦN 2 (THAO TÁC & TRỰC QUAN HÓA)

15.1. Thao tác cơ bản trên DataFrame

Chọn cột/nhiều cột, lọc hàng bằng điều kiện Boolean

15.1.1. Chọn cột/nhiều cột:

- Chọn một cột:
 - Sử dụng dấu ngoặc vuông với tên cột: `df['TênCột']`. Kết quả trả về là một **Series**.
 - Hoặc sử dụng dấu chấm (nếu tên cột không có khoảng trắng hoặc ký tự đặc biệt): `df.TênCột`.

```
import pandas as pd
data = {'Name': ['Bim', 'Bo'], 'Age': [13, 20], 'City': ['HCM', 'TG']}
df = pd.DataFrame(data)
print(df['Name'])
```

Output:

```
# 0    Bim
# 1    Bo
# Name: Name, dtype: object
print(df.Age)
```

Output:

```
# 0    13
# 1    20
# Name: Age, dtype: int64
```

- Chọn nhiều cột: Sử dụng một **list** các tên cột trong dấu ngoặc vuông: `df[['TênCột1', 'TênCột2']]`. Kết quả trả về là một **DataFrame**.

```
print(df[['Name', 'City']])
```

Output:

```
#      Name City
# 0  Bim    HCM
# 1   Bo    TG
```

15.1.2. Lọc hàng bằng điều kiện Boolean (Boolean Indexing):

- Đây là cách cực kỳ mạnh mẽ để chọn các hàng dựa trên giá trị của chúng.
- Bạn tạo ra một **Series Boolean** (chỉ chứa True/False) có cùng độ dài với số hàng của DataFrame, sau đó sử dụng Series này để lọc.

```
# Lọc các hàng mà cột 'Age' lớn hơn 15
df_filtered = df[df['Age'] > 15]
print(df_filtered)
```

Output:

```
#   Name  Age City
# 1   Bo   20  TG
```

```
# Kết hợp nhiều điều kiện sử dụng & (AND) hoặc | (OR)
df_filtered_complex = df[(df['Age'] > 10) & (df['City'] == 'TG')]
print(df_filtered_complex)
```

Output:

```
#   Name  Age City
# 0   Bo   15  TG
```

- **Lưu ý:** Luôn bao các điều kiện riêng lẻ trong ngoặc đơn khi kết hợp.

15.1.3. Thêm/xóa cột mới

1./ Thêm cột mới:

- Gán một Series hoặc một giá trị cho một tên cột mới:

```
# Thêm cột 'Country' với giá trị mặc định
df['Country'] = 'VN'
print(df)
```

Output:

```
#   Name  Age City Country
# 0  Bim   13  HCM      VN
# 1   Bo   20  TG      VN
```

```
# Thêm cột 'Status' dựa trên điều kiện của cột 'Age'
df['Status'] = ['Adult' if age >= 10 else 'Minor' for age in df['Age']]
print(df)
```


Output:

```
#      Name  Age City Country Status
# 0  Bim    15   HCM      VN  Adult
# 1   Bo    20   TG      UVN  Adult
```

2./ Xóa cột:

- Sử dụng phương thức `df.drop()` với `axis=1` (để chỉ ra rằng bạn đang xóa cột) và `inplace=True` nếu muốn thay đổi trực tiếp DataFrame gốc.
- Hoặc sử dụng `del df['TênCột']`.

```
df_new = df.copy() # Tạo bản sao để không thay đổi df gốc
# Xóa cột 'Status'
df_new = df_new.drop('Status', axis=1) # Trả về DataFrame mới
print(df_new)
# Hoặc xóa tại chỗ
# df_new.drop('Status', axis=1, inplace=True)
# Xóa nhiều cột
# df_new.drop(['City', 'Country'], axis=1, inplace=True)
```

15.1.4. Sắp xếp dữ liệu (`sort_values()`)

- Sử dụng phương thức `df.sort_values()` để sắp xếp DataFrame theo giá trị của một hoặc nhiều cột.

```
df_sorted = df.sort_values(by='Age', ascending=False) # Sắp xếp giảm dần
theo 'Age'
print(df_sorted)
```

Output:

```
#      Name  Age City Country Status
# 1   Bo    30   LD      USA  Adult
# 0  Bim    25   NY      USA  Adult

# Sắp xếp theo nhiều cột
# df.sort_values(by=['City', 'Age'], ascending=[True, False])
```

- `by`: Tên cột hoặc danh sách tên cột để sắp xếp.
- `ascending`: True cho tăng dần, False cho giảm dần. Có thể là một Boolean hoặc một list các Boolean nếu sắp xếp theo nhiều cột.
- `inplace=True`: Thay đổi trực tiếp DataFrame.

15.1.5. Nhóm dữ liệu (groupby()) và các phép tổng hợp (sum, mean, count)

- `df.groupby('TênCột')`: Nhóm các hàng có cùng giá trị trong một cột cụ thể. Kết quả trả về là một đối tượng `DataFrameGroupBy`.
- Sau khi nhóm, bạn có thể áp dụng các **hàm tổng hợp (aggregation functions)** như `sum()`, `mean()`, `count()`, `min()`, `max()`, `std()` (độ lệch chuẩn), `median()` (trung vị), v.v.

```
data = {'Category': ['A', 'B', 'A', 'B', 'A'],
        'Value': [10, 20, 15, 25, 12]}
df_group = pd.DataFrame(data)

# Tính tổng 'Value' theo từng 'Category'
grouped_sum = df_group.groupby('Category')['Value'].sum()
print(f"Tổng theo Category:\n{grouped_sum}\n")
```

Output:

```
# Category
# A      37
# B      45
# Name: Value, dtype: int64

# Tính trung bình 'Value' theo từng 'Category'
grouped_mean = df_group.groupby('Category')['Value'].mean()
print(f"Trung bình theo Category:\n{grouped_mean}\n")
```

Output:

```
# Category
# A      12.333333
# B      22.500000
# Name: Value, dtype: float64

# Tính số lượng phần tử theo từng 'Category'
grouped_count = df_group.groupby('Category')['Value'].count()
print(f"Số lượng theo Category:\n{grouped_count}\n")
```

Output:

```
# Category
# A      3
# B      2
# Name: Value, dtype: int64
```

```
# Áp dụng nhiều hàm tổng hợp cùng lúc
grouped_multi_agg = df_group.groupby('Category')['Value'].agg(['sum',
'mean', 'min', 'max'])
print(f"Nhiều tổng hợp theo Category:\n{grouped_multi_agg}\n")
```

Output:

#	sum	mean	min	max
# Category				
# A	37	12.333333	10	15
# B	45	22.500000	20	25

15.2. Xử lý dữ liệu thiếu & Trực quan hóa cơ bản

15.2.1. Xử lý dữ liệu thiếu (Missing Data)

Dữ liệu thiếu (Missing Data), thường được biểu diễn bằng NaN (Not a Number) trong Pandas, là một vấn đề phổ biến và cần được xử lý trước khi phân tích.

1./ Kiểm tra dữ liệu thiếu:

- `df.isnull()` hoặc `df.isna()`: Trả về một DataFrame Boolean cùng kích thước, với True ở những vị trí có giá trị thiếu.
- `df.isnull().sum()`: Đếm số lượng giá trị thiếu trong mỗi cột. Rất hữu ích.
- `df.isnull().sum().sum()`: Đếm tổng số giá trị thiếu trong toàn bộ DataFrame.
- `df.notnull()`: Ngược lại với `isnull()`.

```
import numpy as np
data_nan = {'A': [1, 2, np.nan], 'B': [4, np.nan, 6], 'C': [7, 8, 9]}
df_nan = pd.DataFrame(data_nan)
print(f"DataFrame với dữ liệu thiếu:\n{df_nan}\n")

print(f"Kiểm tra NaN:\n{df_nan.isnull()}\n")
print(f"Số lượng NaN mỗi cột:\n{df_nan.isnull().sum()}\n")
print(f"Tổng số NaN trong DataFrame: {df_nan.isnull().sum().sum()}\n")
```

2./ Xóa dữ liệu thiếu (`dropna()`):

- `df.dropna(axis=0, how='any', inplace=False)`: Xóa các hàng hoặc cột chứa giá trị thiếu.
 - `axis=0`: Xóa hàng (mặc định).
 - `axis=1`: Xóa cột.

- `how='any'`: Xóa nếu có **bất kỳ** giá trị thiếu nào (mặc định).
- `how='all'`: Xóa nếu **tất cả** các giá trị trong hàng/cột đều là thiếu.
- `inplace=True`: Thay đổi trực tiếp DataFrame.

```
df_dropped_rows = df_nan.dropna() # Xóa hàng có bất kỳ NaN nào
print(f"DataFrame sau khi xóa hàng có NaN:\n{df_dropped_rows}\n")

df_dropped_cols = df_nan.dropna(axis=1) # Xóa cột có bất kỳ NaN nào
print(f"DataFrame sau khi xóa cột có NaN:\n{df_dropped_cols}\n")
```

3./ Điền giá trị thiếu (`fillna()`):

- `df.fillna(value)`: Điền các giá trị thiếu bằng một giá trị cụ thể.
- `df.fillna(method='ffill')`: Điền bằng giá trị gần nhất phía trước (forward fill).
- `df.fillna(method='bfill')`: Điền bằng giá trị gần nhất phía sau (backward fill).
- Thường dùng để điền bằng giá trị trung bình (`mean()`), trung vị (`median()`), hoặc mode (`mode()`) của cột đó.

```
# Điền NaN bằng 0
df_filled_zero = df_nan.fillna(0)
print(f"DataFrame sau khi điền NaN bằng 0:\n{df_filled_zero}\n")

# Điền NaN bằng giá trị trung bình của cột
mean_A = df_nan['A'].mean()
df_filled_mean_A = df_nan['A'].fillna(mean_A)
print(f"Cột 'A' sau khi điền NaN bằng mean:\n{df_filled_mean_A}\n")

# Điền NaN bằng forward fill
df_filled_ffill = df_nan.fillna(method='ffill')
print(f"DataFrame sau khi điền NaN bằng ffill:\n{df_filled_ffill}\n")
```

15.2.2. Giới thiệu thư viện Matplotlib: Khái niệm Axes, Figure

Matplotlib là thư viện phổ biến nhất để tạo các biểu đồ tĩnh, hoạt hình và tương tác trong Python.

- **Figure**: Là toàn bộ cửa sổ hoặc trang mà biểu đồ được vẽ lên. Nó có thể chứa một hoặc nhiều Axes.
- **Axes**: Là khu vực mà dữ liệu được vẽ lên (trục X và Y, các điểm dữ liệu, đường, v.v.). Một Figure có thể có nhiều Axes (sub-plots).

Quy trình cơ bản để vẽ biểu đồ:

1. Import matplotlib.pyplot (thường là plt).
2. Tạo Figure và Axes (thường là fig, ax = plt.subplots()).
3. Vẽ dữ liệu lên Axes (ax.plot(), ax.bar(), v.v.).
4. Tùy chỉnh biểu đồ (tiêu đề, nhãn, màu sắc).
5. Hiển thị biểu đồ (plt.show()).

15.2.3. Các loại biểu đồ cơ bản với Matplotlib/Seaborn: biểu đồ đường (plot()), biểu đồ cột (bar())

Seaborn là một thư viện trực quan hóa dữ liệu được xây dựng trên Matplotlib, cung cấp giao diện cấp cao hơn để tạo các biểu đồ thống kê đẹp và phức tạp hơn với ít code hơn.

1./ Biểu đồ đường (Line Plot):

- Dùng để hiển thị xu hướng của dữ liệu theo thời gian hoặc một biến liên tục khác.
- Thường dùng cho dữ liệu chuỗi thời gian.

```
import matplotlib.pyplot as plt
import seaborn as sns # Thường dùng kết hợp với Matplotlib
# Dữ liệu mẫu
years = [2010, 2011, 2012, 2013, 2014]
sales = [100, 120, 150, 130, 180]

plt.figure(figsize=(8, 5)) # Kích thước biểu đồ
plt.plot(years, sales, marker='o', linestyle='-', color='blue')
plt.title('Xu Hướng Doanh Số Bán Hàng Qua Các Năm')
plt.xlabel('Năm')
plt.ylabel('Doanh số')
plt.grid(True) # Hiển thị lưới
plt.show()
```

2./ Biểu đồ cột (Bar Plot):

- Dùng để so sánh giá trị giữa các danh mục riêng biệt.
- Chiều cao (hoặc độ dài) của cột biểu thị giá trị.

```
categories = ['A', 'B', 'C', 'D']
values = [23, 45, 56, 12]

plt.figure(figsize=(7, 5))
```

```
plt.bar(categories, values, color='skyblue')
plt.title('Giá trị theo Danh mục')
plt.xlabel('Danh mục')
plt.ylabel('Giá trị')
plt.show()

# Với Seaborn (thường dùng cho DataFrame)
# data_bar = pd.DataFrame({'Category': categories, 'Value': values})
# plt.figure(figsize=(7, 5))
# sns.barplot(x='Category', y='Value', data=data_bar, palette='viridis')
# plt.title('Giá trị theo Danh mục (Seaborn)')
# plt.xlabel('Danh mục')
# plt.ylabel('Giá trị')
# plt.show()
```

3./ Biểu đồ phân tán (Scatter Plot):

- Dùng để hiển thị mối quan hệ (tương quan) giữa hai biến số liên tục.
- Mỗi điểm trên biểu đồ đại diện cho một quan sát, với vị trí theo trục X và Y tương ứng với giá trị của hai biến.

```
# Dữ liệu mẫu: Số giờ học và Điểm thi
hours_studied = [2, 3, 4, 5, 6, 7, 8, 9]
exam_scores = [60, 65, 70, 75, 80, 85, 90, 95]
plt.figure(figsize=(8, 6))
plt.scatter(hours_studied, exam_scores, color='red', marker='o')
plt.title('Mối Quan Hệ Giữa Số Giờ Học và Điểm Thi')
plt.xlabel('Số Giờ Học')
plt.ylabel('Điểm Thi')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

# Với Seaborn (tốt hơn cho DataFrame)
# df_scatter = pd.DataFrame({'Hours_Studied': hours_studied,
# 'Exam_Scores': exam_scores})
# plt.figure(figsize=(8, 6))
# sns.scatterplot(x='Hours_Studied', y='Exam_Scores', data=df_scatter,
# hue='Exam_Scores', size='Exam_Scores', sizes=(20, 200),
# palette='coolwarm')
# plt.title('Mối Quan Hệ Giữa Số Giờ Học và Điểm Thi (Seaborn)')
# plt.xlabel('Số Giờ Học')
# plt.ylabel('Điểm Thi')
# plt.show()
```

4./ Biểu đồ tròn (Pie Chart):

- Dùng để hiển thị tỷ lệ phần trăm của các danh mục khác nhau so với tổng thể.
- Không nên dùng khi có quá nhiều danh mục hoặc khi các phần quá nhỏ.

```
labels = ['Đông', 'Tây', 'Nam', 'Bắc']
sizes = [15, 30, 45, 10] # Tỷ lệ phần trăm hoặc giá trị
explode = (0, 0.1, 0, 0) # "Tách" lát bánh thứ hai (Tây) ra ngoài
plt.figure(figsize=(7, 7))
plt.pie(sizes, explode=explode, labels=labels, autopct='%1.1f%%',
        shadow=True, startangle=90)
plt.title('Phân Bố Dân Số theo Vùng')
plt.axis('equal') # Đảm bảo biểu đồ tròn vẽ là một vòng tròn
plt.show()
```

15.2.4. Tùy chỉnh biểu đồ (tiêu đề, nhãn trục, màu sắc)

Matplotlib và Seaborn cung cấp rất nhiều tùy chọn để tùy chỉnh biểu đồ để chúng trở nên rõ ràng và chuyên nghiệp hơn.

- **Tiêu đề biểu đồ:** `plt.title('Tiêu đề của biểu đồ')` hoặc `ax.set_title('Tiêu đề')`
- **Nhãn trục:** `plt.xlabel('Nhãn trục X'), plt.ylabel('Nhãn trục Y')` hoặc `ax.set_xlabel('Nhãn X'), ax.set_ylabel('Nhãn Y')`
- **Màu sắc:** Sử dụng tham số `color` hoặc `palette` (trong Seaborn).
- **Kích thước biểu đồ:** `plt.figure(figsize=(width, height))` (inch).
- **Lưới (Grid):** `plt.grid(True)` hoặc `ax.grid(True)`.
- **Chú giải (Legend):** `plt.legend(['Tên chuỗi dữ liệu'])` hoặc `ax.legend()`.
- **Kiểu đường, điểm:** `linestyle, marker`.
- **Giới hạn trục:** `plt.xlim(min, max), plt.ylim(min, max)`.
- **Thêm văn bản vào biểu đồ:** `plt.text(x, y, 'Text')`.
- **Lưu biểu đồ:** `plt.savefig('my_plot.png')`.

15.3. Quy trình phân tích

Quá trình phân tích dữ liệu không chỉ là viết code mà là một chu trình lặp đi lặp lại.

1./ Chuẩn bị dữ liệu (Data Preparation):

- **Thu thập:** Đọc dữ liệu từ các nguồn khác nhau (CSV, Excel, Database, API).

- **Khám phá ban đầu:** Sử dụng `head()`, `info()`, `describe()`, `dtypes`, `shape` để hiểu cấu trúc và loại dữ liệu.
- **Làm sạch:** Xử lý dữ liệu thiếu (`fillna`, `dropna`), loại bỏ trùng lặp, sửa lỗi định dạng.
- **Biến đổi (Transformation):** Tạo cột mới, thay đổi kiểu dữ liệu, chuẩn hóa/mã hóa dữ liệu cho phù hợp với phân tích.

2./ Phân tích dữ liệu (Data Analysis):

- **Thống kê mô tả:** Tính toán các giá trị tổng hợp (`mean`, `sum`, `median`, `std`), phân phối tần suất.
- **Phân tích tương quan:** Tìm mối quan hệ giữa các biến.
- **Nhóm và tổng hợp:** Sử dụng `groupby()` để khám phá thông tin từ các nhóm dữ liệu.
- **Tìm kiếm xu hướng và mẫu:** Phát hiện các mẫu định kỳ, các ngoại lệ.

3./ Trực quan hóa dữ liệu (Data Visualization):

- Chọn loại biểu đồ phù hợp với mục tiêu phân tích (đường cho xu hướng, cột cho so sánh, phân tán cho mối quan hệ).
- Vẽ biểu đồ bằng Matplotlib/Seaborn.
- Tùy chỉnh biểu đồ để làm nổi bật thông tin và dễ hiểu.

4./ Giải thích và Báo cáo (Interpretation & Reporting):

- Rút ra kết luận từ các biểu đồ và phân tích thống kê.
- Trả lời các câu hỏi kinh doanh/nghiên cứu ban đầu.
- Trình bày kết quả một cách rõ ràng và mạch lạc, có thể dưới dạng báo cáo, bài thuyết trình, hoặc dashboard.
- Quá trình này thường là lặp đi lặp lại: từ giải thích có thể dẫn đến các câu hỏi mới và quay lại bước chuẩn bị/phân tích.

BÀI TẬP:

1. Xử lý dữ liệu thiếu: tìm số lượng NaN trong mỗi cột, sau đó điền giá trị thiếu của cột số (`Age`, `Score`,...) bằng giá trị trung vị (`median()`).
2. Sử dụng `groupby()` và `.agg()` để tính tổng doanh thu và số lượng đơn hàng đồng thời cho từng `'Region'` và `'Category'`.
3. Vẽ biểu đồ phân tán (Scatter Plot) để kiểm tra mối quan hệ giữa `'Price'` và `'Quantity'` của sản phẩm. Tùy chỉnh biểu đồ với tiêu đề, nhãn trục và chú giải.

THỰC HÀNH 15: THAO TÁC & TRỰC QUAN HÓA

15.1. Thao tác DataFrame

15.1.1. Thực hành lọc dữ liệu theo các điều kiện khác nhau (ví dụ: lọc sản phẩm giá trên 100\$, lọc sinh viên điểm trên 8)

Mục tiêu: Nắm vững cách chọn và lọc dữ liệu trong DataFrame.

Các bước thực hành:

1. Tạo một file Python mới, ví dụ dataframe_manipulation.py.
2. Tạo DataFrame mẫu:

```
import pandas as pd
import numpy as np

data = {
    'Product': ['Laptop', 'Mouse', 'Keyboard', 'Monitor', 'Webcam',
                'Headphones'],
    'Category': ['Electronics', 'Electronics', 'Electronics',
                 'Electronics', 'Accessories', 'Accessories'],
    'Price': [1200, 25, 75, 300, 50, 150],
    'Stock': [10, 200, 150, 20, 80, 60],
    'Rating': [4.5, 4.0, 4.2, 3.9, 4.1, 4.3]
}

df_products = pd.DataFrame(data)
print("DataFrame gốc:\n", df_products)
print("-" * 30)

# 1. Lọc sản phẩm có giá trên 100$
high_price_products = df_products[df_products['Price'] > 100]
print("Sản phẩm có giá > 100$:\n", high_price_products)
print("-" * 30)

# 2. Lọc sản phẩm thuộc danh mục 'Electronics' và có Stock < 50
electronic_low_stock = df_products[(df_products['Category'] ==
                                     'Electronics') & (df_products['Stock'] < 50)]
print("Sản phẩm Điện tử với số lượng tồn kho thấp:\n",
      electronic_low_stock)
print("-" * 30)
```

```
# 3. Lọc sản phẩm có Rating từ 4.2 trở lên HOẶC giá dưới 50$
good_rating_or_cheap = df_products[(df_products['Rating'] >= 4.2) |
(df_products['Price'] < 50)]
print("Sản phẩm có Rating tốt hoặc giá rẻ:\n", good_rating_or_cheap)
print("-" * 30)

# 4. Chọn chỉ các cột 'Product' và 'Price' của các sản phẩm có Stock <
100
selected_cols_filtered = df_products[df_products['Stock'] <
100][['Product', 'Price']]
print("Sản phẩm tồn kho thấp (chỉ Product và Price):\n",
selected_cols_filtered)
print("-" * 30)
```

3. Lưu và chạy file.

15.1.2. Nhóm dữ liệu và tính toán các giá trị tổng hợp (ví dụ: tổng doanh thu theo khu vực, điểm trung bình theo lớp)

Mục tiêu: Sử dụng groupby() để tổng hợp dữ liệu.

Các bước thực hành:

1. Tiếp tục file dataframe_manipulation.py.
2. Thêm đoạn code sau:

```
# Tạo DataFrame mới cho ví dụ groupby
sales_data = {
    'Region': ['East', 'West', 'East', 'Central', 'West', 'East',
'Central'],
    'Product_Type': ['A', 'B', 'A', 'C', 'B', 'A', 'C'],
    'Revenue': [1000, 1500, 1200, 800, 1300, 900, 700],
    'Units_Sold': [10, 15, 12, 8, 13, 9, 7]
}
df_sales = pd.DataFrame(sales_data)
print("DataFrame Doanh số:\n", df_sales)
print("-" * 30)

# 1. Tổng doanh thu theo từng khu vực
revenue_by_region = df_sales.groupby('Region')['Revenue'].sum()
print("Tổng doanh thu theo Khu vực:\n", revenue_by_region)
print("-" * 30)
```

```
# 2. Số lượng đơn vị bán được trung bình theo loại sản phẩm
avg_units_by_product_type =
df_sales.groupby('Product_Type')['Units_Sold'].mean()
print("Đơn vị bán trung bình theo Loại sản phẩm:\n",
avg_units_by_product_type)
print("-" * 30)

# 3. Tổng hợp nhiều thống kê (tổng, trung bình, min, max) theo khu vực và
loại sản phẩm
multi_agg = df_sales.groupby(['Region', 'Product_Type']).agg(
    Total_Revenue=('Revenue', 'sum'),
    Average_Units=('Units_Sold', 'mean'),
    Min_Revenue=('Revenue', 'min')
)
print("Tổng hợp nhiều thống kê theo Khu vực và Loại sản phẩm:\n",
multi_agg)
print("-" * 30)
```

3. Lưu và chạy file.

15.1.3. Thêm một cột mới dựa trên các cột hiện có

Mục tiêu: Tạo các trường dữ liệu dẫn xuất.

Các bước thực hành:

1. Tiếp tục file dataframe_manipulation.py.
2. Thêm đoạn code sau:

```
# Quay lại df_products
print("DataFrame sản phẩm gốc:\n", df_products)
print("-" * 30)

# 1. Thêm cột 'Total_Value' = Price * Stock
df_products['Total_Value'] = df_products['Price'] * df_products['Stock']
print("DataFrame với cột 'Total_Value':\n", df_products)
print("-" * 30)

# 2. Thêm cột 'Is_High_Rated' (True nếu Rating >= 4.2)
df_products['Is_High_Rated'] = df_products['Rating'] >= 4.2
print("DataFrame với cột 'Is_High_Rated':\n", df_products)
print("-" * 30)
```

```
# 3. Thêm cột 'Discount_Price' (giảm 10% nếu Total_Value > 10000)
df_products['Discount_Price'] = np.where(df_products['Total_Value'] >
10000,
                                         df_products['Price'] * 0.9,
                                         df_products['Price'])
print("DataFrame với cột 'Discount_Price':\n", df_products)
print("-" * 30)
```

- np.where(condition, value_if_true, value_if_false) là một hàm NumPy hữu ích để tạo cột mới dựa trên điều kiện.

3. Lưu và chạy file.

15.2. Xử lý dữ liệu thiếu & Biểu đồ cơ bản

15.2.1. Kiểm tra và xử lý dữ liệu thiếu trong DataFrame (thực hành dropna() và fillna())

Mục tiêu: Làm sạch dữ liệu thiếu.

Các bước thực hành:

1. Tạo một file Python mới, ví dụ missing_data_plots.py.
2. Tạo DataFrame với dữ liệu thiếu:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Dữ liệu mẫu với NaN
data_missing = {
    'ID': [1, 2, 3, 4, 5, 6],
    'Name': ['Bim', 'Bo', 'Bin', 'Bi', np.nan, 'Bống'],
    'Score': [85, 90, np.nan, 78, 92, 88],
    'City': ['NY', 'LA', 'NY', np.nan, 'LA', 'NY']
}
df_missing = pd.DataFrame(data_missing)
print("DataFrame gốc với dữ liệu thiếu:\n", df_missing)
print("-" * 30)

# 1. Kiểm tra dữ liệu thiếu
print("Số lượng giá trị thiếu mỗi cột:\n", df_missing.isnull().sum())
print("-" * 30)
```

```
# 2. Xóa các hàng có bất kỳ giá trị thiếu nào
df_after_dropna = df_missing.dropna()
print("DataFrame sau khi xóa hàng có NaN:\n", df_after_dropna)
print("-" * 30)

# 3. Điền giá trị thiếu trong cột 'Score' bằng giá trị trung bình của cột đó
mean_score = df_missing['Score'].mean()
df_filled_score = df_missing.copy() # Làm việc trên bản sao
df_filled_score['Score'] = df_filled_score['Score'].fillna(mean_score)
print(f"Điền NaN trong 'Score' bằng trung bình ({mean_score:.2f}):\n",
      df_filled_score)
print("-" * 30)

# 4. Điền giá trị thiếu trong cột 'Name' và 'City' bằng một giá trị cố
định (ví dụ: 'Unknown')
df_filled_categorical = df_missing.copy()
df_filled_categorical['Name'] =
df_filled_categorical['Name'].fillna('Unknown')
df_filled_categorical['City'] =
df_filled_categorical['City'].fillna('Unknown')
print("Điền NaN trong 'Name' và 'City' bằng 'Unknown':\n",
      df_filled_categorical)
print("-" * 30)

# 5. Điền giá trị thiếu bằng phương pháp forward-fill (ffill)
df_ffill = df_missing.fillna(method='ffill')
print("Điền NaN bằng ffill:\n", df_ffill)
print("-" * 30)
```

3. Lưu và chạy file.

15.2.2. Vẽ biểu đồ đường để thể hiện xu hướng thời gian

Mục tiêu: Trực quan hóa xu hướng dữ liệu.

Các bước thực hành:

1. Tiếp tục file `missing_data_plots.py`.
2. Thêm đoạn code sau:

```
# Dữ liệu mẫu chuỗi thời gian
time_series_data = {
    'Month': ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun'],
```

```
'Revenue': [100, 120, 110, 140, 130, 160],
'Expenses': [70, 75, 72, 80, 78, 85]
}
df_time_series = pd.DataFrame(time_series_data)

print("DataFrame chuỗi thời gian:\n", df_time_series)
print("-" * 30)

plt.figure(figsize=(10, 6))
plt.plot(df_time_series['Month'], df_time_series['Revenue'], marker='o',
linestyle='-', color='blue', label='Doanh thu')
plt.plot(df_time_series['Month'], df_time_series['Expenses'], marker='x',
linestyle='--', color='red', label='Chi phí')

plt.title('Xu Hướng Doanh thu và Chi phí theo Tháng')
plt.xlabel('Tháng')
plt.ylabel('Giá trị ($)')
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend() # Hiển thị chú giải
plt.show()
```

3. Lưu và chạy file.

15.2.3. Vẽ biểu đồ cột để so sánh các danh mục

Mục tiêu: So sánh giá trị giữa các nhóm.

Các bước thực hành:

1. Tiếp tục file `missing_data_plots.py`.
2. Thêm đoạn code sau:

```
# Dữ liệu mẫu danh mục
category_data = {
    'Department': ['HR', 'IT', 'Sales', 'Marketing', 'Finance'],
    'Employees': [40, 120, 90, 60, 50],
    'Budget_M': [2.5, 8.0, 6.5, 4.0, 3.5]
}
df_categories = pd.DataFrame(category_data)

print("DataFrame Danh mục:\n", df_categories)
print("-" * 30)
```

```
# Biểu đồ cột số lượng nhân viên theo phòng ban
plt.figure(figsize=(9, 6))
sns.barplot(x='Department', y='Employees', data=df_categories,
palette='viridis')
plt.title('Số Lượng Nhân Viên theo Phòng Ban')
plt.xlabel('Phòng Ban')
plt.ylabel('Số Lượng Nhân Viên')
plt.grid(axis='y', linestyle='--', alpha=0.7) # Chỉ hiển thị lưới trục y
plt.show()

# Biểu đồ cột ngân sách theo phòng ban
plt.figure(figsize=(9, 6))
sns.barplot(x='Department', y='Budget_M', data=df_categories,
palette='rocket')
plt.title('Ngân Sách theo Phòng Ban (Triệu USD)')
plt.xlabel('Phòng Ban')
plt.ylabel('Ngân Sách (Triệu USD)')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

3. Lưu và chạy file.

15.2.4. Vẽ biểu đồ phân tán để khám phá mối quan hệ giữa hai biến

Mục tiêu: Trực quan hóa mối quan hệ giữa hai biến số liên tục.

Các bước thực hành:

1. Tiếp tục file `missing_data_plots.py`.
2. Thêm đoạn code sau:

```
# Dữ liệu mẫu: Chiều cao và Cân nặng
physical_data = {
    'Height_cm': [160, 165, 170, 175, 180, 185, 190, 155],
    'Weight_kg': [55, 60, 65, 70, 75, 80, 85, 50],
    'Gender': ['F', 'F', 'M', 'M', 'M', 'M', 'M', 'F']
}
df_physical = pd.DataFrame(physical_data)

print("DataFrame Chiều cao - Cân nặng:\n", df_physical)
print("-" * 30)

plt.figure(figsize=(9, 7))
```

```
sns.scatterplot(x='Height_cm', y='Weight_kg', data=df_physical,
hue='Gender', s=100, alpha=0.8, palette='coolwarm')
plt.title('Biểu đồ Phân tán Chiều cao và Cân nặng')
plt.xlabel('Chiều cao (cm)')
plt.ylabel('Cân nặng (kg)')
plt.grid(True, linestyle=':', alpha=0.6)
plt.show()
```

3. Lưu và chạy file.

15.3. Mini-project phân tích dữ liệu

Mục tiêu: Áp dụng toàn bộ quy trình phân tích dữ liệu cơ bản vào một tập dữ liệu thực tế.

Các bước thực hành:

1. Chọn/Tải dữ liệu:

- Bạn có thể sử dụng file sales_data.csv đã tạo ở tuần trước, hoặc tìm một file CSV/Excel nhỏ khác.
- Ví dụ sử dụng sales_data.csv đã được thêm cột Avg_Revenue_Per_Unit ở tuần trước (hoặc bạn có thể thêm nó ngay trong bước này).

2. Tạo file mới cho mini-project: Tạo file mini_project.py.

3. Viết code thực hiện các bước:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# --- 1. Tải/đọc dữ liệu ---
# Giả sử bạn có file 'sales_data.csv' trong cùng thư mục
try:
    df_project = pd.read_csv('sales_data.csv')
    print("Đã đọc dữ liệu thành công.")
except FileNotFoundError:
    print("Lỗi: Không tìm thấy file 'sales_data.csv'. Vui lòng kiểm tra đường dẫn hoặc tạo file.")
    exit() # Thoát nếu không tìm thấy file

print("\n--- Khám phá dữ liệu ban đầu ---")
print("5 hàng đầu tiên:\n", df_project.head())
print("\nThông tin dữ liệu (info()):")
```



```
df_project.info()
print("\nThông kê mô tả (describe()):")
print(df_project.describe())
print("\nSố lượng giá trị thiếu:\n", df_project.isnull().sum())
# --- 2. Làm sạch sơ bộ (nếu cần) ---
# Trong ví dụ này, giả sử không có NaN hoặc dữ liệu thiếu được xử lý ở đây
# Nếu có, bạn có thể thêm:
# df_project.dropna(inplace=True)
# df_project['Column_Name'].fillna(df_project['Column_Name'].mean(),
# inplace=True)
# Chuyển đổi cột 'Date' sang kiểu datetime để dễ xử lý thời gian
df_project['Date'] = pd.to_datetime(df_project['Date'])
print("\nDataFrame sau khi chuyển đổi kiểu dữ liệu 'Date':\n",
df_project.head())
# --- 3. Thực hiện một phân tích đơn giản ---
# a. Tính tổng doanh thu toàn bộ
total_revenue = df_project['Sales'].sum()
print(f"\nTổng doanh thu toàn bộ: ${total_revenue:,.2f}")
# b. Tính tổng doanh thu theo từng khu vực
revenue_by_region =
df_project.groupby('Region')['Sales'].sum().sort_values(ascending=False)
print("\nTổng doanh thu theo Khu vực:\n", revenue_by_region)
# c. Tìm sản phẩm bán chạy nhất
product_sales =
df_project.groupby('Product')['Sales'].sum().sort_values(ascending=False)
print("\nTổng doanh thu theo Sản phẩm:\n", product_sales)
top_product = product_sales.index[0]
top_product_revenue = product_sales.iloc[0]
print(f"\nSản phẩm bán chạy nhất là '{top_product}' với doanh thu:
${top_product_revenue:,.2f}")
# d. Thêm cột 'Profit' (giả định 30% doanh thu là lợi nhuận)
df_project['Profit'] = df_project['Sales'] * 0.3
print("\nDataFrame với cột 'Profit':\n", df_project.head())

# --- 4. Trực quan hóa kết quả bằng ít nhất 2 loại biểu đồ ---
# Biểu đồ 1: Biểu đồ cột tổng doanh thu theo khu vực
plt.figure(figsize=(8, 5))
sns.barplot(x=revenue_by_region.index, y=revenue_by_region.values,
palette='viridis')
plt.title('Tổng Doanh thu theo Khu vực')
```

```
plt.xlabel('Khu vực')
plt.ylabel('Doanh thu ($)')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout() # Điều chỉnh layout để tránh cắt xén nhãn
plt.show()

# Biểu đồ 2: Biểu đồ đường xu hướng doanh số theo thời gian (nếu có đủ
điểm dữ liệu)
# Group by date for daily sales
daily_sales = df_project.groupby('Date')['Sales'].sum()
plt.figure(figsize=(10, 6))
plt.plot(daily_sales.index, daily_sales.values, marker='o', linestyle='-',
        color='green')
plt.title('Xu Hướng Doanh số theo Ngày')
plt.xlabel('Ngày')
plt.ylabel('Doanh số ($)')
plt.grid(True, linestyle='--', alpha=0.6)
plt.xticks(rotation=45) # Xoay nhãn trục x cho dễ đọc
plt.tight_layout()
plt.show()

# Biểu đồ 3 (Tùy chọn): Biểu đồ tròn phân bố doanh số theo loại sản phẩm
# product_sales_pie = df_project.groupby('Product')['Sales'].sum()
# plt.figure(figsize=(8, 8))
# plt.pie(product_sales_pie, labels=product_sales_pie.index,
#         autopct='%1.1f%%', startangle=90, colors=sns.color_palette('pastel'))
# plt.title('Tỷ lệ Doanh thu theo Loại Sản phẩm')
# plt.axis('equal')
# plt.show()

# --- 5. Đưa ra một vài nhận xét/kết luận từ phân tích ---
print("\n--- Nhận xét/Kết luận ---")
print(f"- Khu vực '{revenue_by_region.index[0]}' có doanh thu cao nhất,
cho thấy đây là thị trường tiềm năng nhất.")
print(f"- Sản phẩm '{top_product}' đang đóng góp doanh thu lớn nhất.")
print(f"- Dữ liệu cho thấy xu hướng doanh số có sự biến động theo ngày,
cần phân tích sâu hơn về các yếu tố tác động (ví dụ: ngày trong tuần,
chương trình khuyến mãi).")
print(f"- Cột 'Profit' mới tạo giúp ước tính lợi nhuận từ mỗi giao dịch.")
```

4. Lưu và chạy file.